



Research Paper

RT-APT: A real-time APT anomaly detection method for large-scale provenance graph

Zhengqiu Weng^a, Weinuo Zhang^a, Tiantian Zhu^b, Zhenhao Dou^a, Haofei Sun^b, Zhanxiang Ye^{c,d,*}, Ye Tian^b

^a School of Data Science and Artificial Intelligence, Wenzhou University of Technology, Wenzhou, 325035, China

^b College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, 310023, China

^c School of Artificial Intelligence, Wenzhou Polytechnic, Wenzhou, 325035, China

^d Wenzhou Cyber Security and Protection Technology Research Center, Wenzhou, 325035, China

ARTICLE INFO

Handling Editor: M. Atiquzzaman

Keywords:

APT attack

Provenance graph

Anomaly detection

Clustering analysis

ABSTRACT

Advanced Persistent Threats (APTs) are prevalent in the field of cyber attacks, where attackers employ advanced techniques to control targets and exfiltrate data without being detected by the system. Existing APT detection methods heavily rely on expert rules or specific training scenarios, resulting in the lack of both generality and reliability. Therefore, this paper proposes a novel real-time APT attack anomaly detection system for large-scale provenance graphs, named RT-APT. Firstly, a provenance graph is constructed with kernel logs, and the WL subtree kernel algorithm is utilized to aggregate contextual information of nodes in the provenance graph. In this way we obtain vector representations. Secondly, the FlexSketch algorithm transforms the streaming provenance graph into a sequence of feature vectors. Finally, the K-means clustering algorithm is performed on benign feature vector sequences, where each cluster represents a different system state. Thus, we can identify abnormal behaviors during system execution. Therefore RT-APT enables to detect unknown attacks and extract long-term system behaviors. Experiments have been carried out to explore the optimal parameter settings under which RT-APT can perform best. In addition, we compare RT-APT and the state-of-the-art approaches on three datasets, Laboratory, StreamSpot and Unicorn. Results demonstrate that our proposed method outperforms the state-of-the-art approaches from the perspective of runtime performance, memory overhead and CPU usage.

1. Introduction

In the past decades, different from traditional attacks, a novel type of attacks has become increasingly popular. These attacks are typically carried out by organized and well-resourced attackers who employ slow and low-profile techniques to compromise targets, often specific organizations, government agencies, or commercial enterprises, in order to steal data without being detected by defense systems. Such activities are referred to as Advanced Persistent Threats (APTs). APT attackers access the networks of their target entities, using known application vulnerabilities, malware, spear-phishing, zero-day exploits, network downloads, watering hole attacks, etc. According to the “Global Advanced Persistent Threat Mid-year Report 2023¹”, government sectors remain primary targets of APT attacks, accounting for 30% of related world-

wide incidents, followed by defense and military sectors with a 16% share. APT attacks often operate stealthily. Attackers hide in the compromised systems for extended periods, where they clandestinely gather information, or conduct malicious operations. Currently, for unknown APT attacks, there are significant challenges.

A significant amount of research has been conducted by numerous experts and scholars on defending against APTs. Recent research points out that AI techniques can be applied in the field of cybersecurity (Yu et al., 2020). In the field of multi-agent systems, researchers have explored cooperative fault-tolerant output regulation through adaptive dynamic event-triggered mechanisms (Piskozub et al., 2021). In addition, the H_∞ controller synthesis technique (Kang et al., 2022) shows its effectiveness when dealing with network systems with mixed attacks and disturbances, which provides an important reference for our APT

* Corresponding author. School of Artificial Intelligence, Wenzhou Polytechnic, Wenzhou, 325035, China.

E-mail addresses: derisweng@wzut.edu.cn (Z. Weng), limitixynuo@gmail.com (W. Zhang), tzhu@zjut.edu.cn (T. Zhu), 221122120316@zjut.edu.cn (H. Sun), wzyznetwork@wzpt.edu.cn (Z. Ye), 2112012221@zjut.edu.cn (Y. Tian).

¹ <https://www.qianxin.com/threat/reportdetail?reportid=295>.

<https://doi.org/10.1016/j.jnca.2024.104036>

Received 2 May 2024; Received in revised form 10 August 2024; Accepted 24 September 2024

Available online 10 October 2024

1084-8045/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

detection system. However, APT attacks have different characteristics from traditional attack patterns, and conventional detection systems cannot effectively identify APT attacks. Previous research has focused on analyzing system-originated data, but ignored two elements: generality and reliability, which needs to be considered in our proposed APT detection system. We detail these two elements in detail.

- (1) **Generality.** APTs feature in their prolonged duration, lengthy attack chains, high-level stealthiness, diverse techniques, etc. Traditional single-point intrusion detection based countermeasures cannot effectively address them. For instance, to circumvent the attacks, the attackers can use traffic encryption for the attack detection based on network traffic analysis (e.g., OWL (Song et al., 2021), SH-LOT (Pan et al., 2018), MalPhase (Shi et al., 2020)) and polymorphic variations and obfuscation of attack samples for the attack detection based on software static feature analysis (e.g., TRACER (Qu et al., 2017)). The dynamic sandbox analysis (e.g., VPBox (Xiong et al., 2020), FlowCog (Hossain et al., 2017)) is not suitable for client-side deployments due to significant overhead, and attackers often employ anti-detection techniques to mislead sandbox behaviors. Detection technologies based on hook tracking analysis (e.g., VAHunt (Milajerdi et al., 2019a), Appshield (Zhu et al., 2023)) modify the operating system kernel, which has been prohibited in Windows operating systems because it can adversely affect system stability. Although rule-based matching methods (e.g., CONAN (Feng et al., 2003), SLEUTH (Maggi et al., 2008), HOLMES (Mutz et al., 2007)) can detect many attacks, they heavily rely on predefined rules or patterns, requiring extensive human involvement, and it often proves that they does not work in unknown attacks, such as 0-day vulnerabilities. Moreover, the rule formulation in previous detection systems has been cumbersome as the rules are designed for specific environments. For instance, APTSHIELD (Manzoor et al., 2016) formulates rules based on the contextual semantics within the origin graph, constructs system event compression strategies that maintain global dependencies and preserve attack semantics, APTSHIELD also conducts APT attack response and alerting through label transfer and aggregation of reduced data. However, it is difficult for the system to detect new attacks and adapt to changing environments.
- (2) **Reliability.** APT attacks are characterized by their long duration and strong concealment, and are hard to detect with context information and provenance graphs. Researchers face such issues as a broad period of attacks, many attack-irrelevant events, and attack-relevant problems. Existing short sequence detection systems (Han et al., 2020) (Kapoor et al., 2021) (Zengy et al., 2022) perform poor in the long-term characteristics of APT attacks, as they fail to model and analyze the long-term behaviors of the system. Moreover, they are susceptible to evasion techniques and prone to mimicry attacks. StreamSpot (Chen et al., 2022) and Unicorn (Milajerdi et al., 2019b) use long sequence events for anomaly detection and utilize automata to model how the system state changes. PROV-GEM (Hassan et al., 2020) transforms graphs into aggregated node embeddings to capture context information and adopts supervised learning for classification. ShadeWatcher (Hassan et al., 2019) detects malicious interactions between entities by constructing a recommendation system based on Graph Neural Networks (GNNs). APT-KGL (Hossain et al., 2020) proposes a GNN-based intelligent APT detection method combining heterogeneous graph embeddings pre-training to reduce real-time processing complexity and determine the detection scope through subgraph sampling. However, existing graph-based modeling approaches have a coarse granularity, requiring analysts to spend significant time identifying malicious entities or events from subgraphs, which may potentially contain thousands of nodes. Furthermore, it is

hard to train the detection models with high generalization, since the diverse and numerous attack-related samples required in training cannot be obtained from the low-frequency APT attacks. In long-term attack scenarios, maintaining incremental system behavior embeddings incurs significant costs and introduces the concept drift problem, thus affecting detection reliability.

This paper proposes a novel real-time detection system for large-scale provenance graph, named RT-APT. Firstly, this method begins with the analysis of kernel logs from the operating system, which meticulously record system calls and kernel module activities, providing us with rich contextual information. We have chosen fine grained logging that captures subtle system behaviors such as file operations and network connections laying the groundwork for the construction of the provenance graph. The log data is collected through the SPADE tool and then transformed by a preprocessing module into data structures for nodes and edges, which are subsequently used to build the provenance graph. In the provenance graph, each node represents an entity within the system, such as a process, file, or network connection, while edges denote the interactions or relationships between these entities. This approach allows us to capture and represent the system's execution history and the complex causal relationships between entities in detail. The rich contextual information of nodes in the provenance graph is then aggregated with the WL subtree graph kernel algorithm, and these nodes are transformed into vector representations. Secondly, the FlexSketch algorithm is employed to handle the streaming growth of the provenance graph, generating a feature vector sequence that maintains similarity and allows for incremental updates. Finally, all benign feature vector sequences are clustered, and each cluster represents various states of system execution. Anomalous behaviors in system execution are detected by using the K-means clustering-based detection method.

In this study, the RT-APT system proposed offers advanced detection capabilities for APT attacks across various domains in the real world. The real-time anomaly detection mechanism is of significant importance for enterprises in protecting sensitive data, for government agencies in defending against cyber espionage activities, and for industrial control systems in preventing potential disruptions. Furthermore, with the proliferation of cloud computing services, the deployment of the RT-APT system can provide an additional layer of security for cloud platforms, safeguarding their user's data and operations from the impact of advanced persistent threats.

This paper makes the following contributions.

- (1) We propose a novel real-time APT attack anomaly detection system (RT-APT), which is different from existing APT detection methods due to that RT-APT considers the balance between generality and reliability. We construct a provenance graph first, then use the WL subtree kernel algorithm (Hodge and Austin, 2004) and Flexsketch algorithm to obtain the contextual information of nodes and the streaming growth respectively, and eventually analyze the long-term behaviors of the system.
- (2) We cluster generated benign feature vector sequences where each cluster represents different states of system execution. During deployment, the K-means clustering-based detection method matches all trained models and detects anomalies in system execution. In this way we can detect the unknown attacks.
- (3) The method proposed, RT-APT, can detect APT attacks with high precision and accuracy, and achieves better detection results on StreamSpot (Chen et al., 2022) dataset and Unicorn dataset.

2. Related work

In this section, to grasp the research status of APT detection, we analyze the provenance graph based detection first and then focus on the graph-based similarity comparison.

2.1. Provenance graph-based APT attack detection

In this subsection, we divide the provenance graph based APT attack detection into heuristic-based detection (rule or pattern matching) and anomaly-based detection.

Pattern-matching based APT attack detection (PM-APT). With network security knowledge and threat intelligence, existing studies manually design predefined rules to detect APT attacks. SLEUTH (Maggi et al., 2008) was the first to reconstruct APT attacks by using provenance graphs, marking audit log information, and constructing dependency graphs. The attack scenarios were reconstructed through label analysis and by defining rules to match threat attacks. Poirot (Berrada et al., 2020), based on the previous work, introduced threat intelligence and added an alignment algorithm for query graphs and provenance graphs. APT attacks were detected by modeling threat detection as an imprecise graph pattern matching (GPM) problem and matching specific subgraphs. Addressing the semantic loss, HOLMES (Mutz et al., 2007) integrated high-level scenario graphs (HSG) and the ATT&CK framework. The former is designed to map the information from the low level to the high level, and the latter is used to map the APT activity information to the kill chain model. In this way, eventually benign or attack scenarios were detected. In addition, PM-APT commonly faces dependency explosion, where many false dependencies result in alert fatigue for analysts. RapSheet (Zimba et al., 2020) and NODOZE (Akoglu et al., 2015) incorporated alerts triggered by the matching of Tactics, Techniques, and Procedures (TTP) rules. The former reduced the long-term system log storage cost. In contrast, the latter effectively reduced the false positive rate and resolved alert fatigue through a path-based threat scoring scheme. MORSE (Ding et al., 2012) employed label decay techniques to mitigate the dependency explosion, that is, reducing the impact of potential threats with increasing propagation rounds. PM-APT is the mainstream approach in the industry due to its high accuracy, strong interpretability, and ease of deployment. However, there are still some limitations. (1) In the rule-matching detection, the comprehensive expertise in network security, operating systems and more is a high design threshold for the security personnel. (2) With the release of malware variants, there is no one-for-all formulated rules. The rules just work for specific scenarios. When the scenarios change or new/unknown attacks emerge, they do not work well.

Anomaly-based attack detection. APT attacks can adapt to defense techniques. Therefore, APT attackers can evade these defense methods in such ways as creating new malware and modifying existing malware to have unique signatures. Hodge and Austin (Liu et al., 2005) investigated different anomaly-based detection methods. They are categorized into three types: unsupervised clustering, which processes data into static distributions, accurately identifies the most remote points, and labels them as potential anomalies; supervised classification, which pre-labels data classified as normal or abnormal; and semi-supervised detection, which models pre-classified data with normal or a small amount of anomalous data. In the unsupervised clustering, when new data arrives, the model is adjusted to improve outlier detection rates by defining a normal boundary. Berrada et al. (Gao et al., 2010) extracted boolean features from provenance graphs and treated APT attack detection as an anomaly detection task. Unlike supervised classification, it does not require abnormal training data but learns to recognize anomalies. StreamSpot (Chen et al., 2022) was an anomaly detection system that employs clustering to handle heterogeneous streaming graphs. It extracted local graph features from individual node/edge labels through a breadth-first traversal and converted them into vectors for classification. In a semi-supervised learning framework, Zimba et al. (Perozzi et al., 2014) identified hosts exhibiting suspicious, malicious activities, where features were extracted based on network clustering algorithms, and a detection model was trained using both labeled and unlabeled data. Han et al. (Milajerdi et al., 2019b) proposed a

machine-learning-based attack detection algorithm, Unicorn, which automatically constructed normal system behaviors and use the clustering to detect abnormal behaviors. PROGRAPHER (Vishwanathan et al., 2010) combines graph embeddings (graph2vec) with sequence learning (TextRCNN), and AIRTAG (Papadimitriou et al., 2010) integrates BERT with a one-class support vector machine (OC-SVM) to achieve fine grained log detection. Alsajri and Steiti (Alshamrani et al., 2019) present a proposed method for detecting intrusion in network traffic using a hybrid approach, which combines a genetic algorithm and an SVM algorithm. However, these methods still face challenges in capturing the long-term behaviors and subtle patterns indicative of APTs.

In comparison, anomaly detection based learning techniques have the following advantages. (1) They can automatically extract potential patterns and construct detection models from training datasets. (2) The detection models can adapt to new environments through automatic retraining or fine-tuning, overcoming the limitations in detecting stealthy attacks.

2.2. Graph feature extraction

Graph similarity calculation aims to determine the degree of similarity between two graphs. It is crucial in various real-world applications, such as object tracking, recommendation systems, binary code analysis, and similarity search. Additionally, it has significant implications in the network security as it helps defense systems issue timely threat alerts and assists cybersecurity analysts in analyzing attacks within a system. Meanwhile, similarity evaluation methods for anomaly detection on network graphs have also become widespread. Akoglu et al. (Berlingerio et al., 2012) classified graphs or subgraphs based on attributes (static vs dynamic, attribute-less vs attributed) to determine their similarity for anomaly detection. Ding et al. (Gehani and Tariq, 2012) identified malicious network sources in flow graphs using vertex-cutting and employed similarity measures like centrality to detect inter-community communication behaviors. Liu et al. (Park and Kim, 2021) constructed a software behavior graph to describe program execution and used support vector machines based on closed and frequent subgraphs to classify non-crash errors (logical errors that do not cause program crashes). However, almost all the graph mining algorithms (Berlingerio et al., 2012) (Gehani and Tariq, 2012) (Park and Kim, 2021) (Shervashidze et al., 2011) (Hunt et al., 2010) and graph similarity measures (e.g., graph kernels (Weisfeiler and Leman, 1968)) are designed for static graphs. There are some challenges when they are applied to similarity comparison on streaming graphs. Therefore, Papadimitriou et al. (Baker and McCallum, 1998) proposed five approaches for similarity comparison on dynamic network graphs (Chapelle et al., 1999). However, the aforementioned similarity evaluation methods (Berlingerio et al., 2012) (Gehani and Tariq, 2012) (Park and Kim, 2021) (Shervashidze et al., 2011) (Baker and McCallum, 1998) (Hunt et al., 2010) (Weisfeiler and Leman, 1968) primarily rely on the existing graph structure and do not involve learning processes, making it difficult to effectively utilize the features provided by nodes and edges. To address this, NetSimile (Xu et al., 2018) leveraged distribution moments to aggregate egonet-based features (such as the number of neighboring nodes) for clustering social networks. Nonetheless, most methods for streaming graphs are often application-specific or only applicable to homogeneous graphs.

In contrast to previous work, this paper utilizes a locality-sensitive hashing (LSH) approach. It employs a consistent weighted sampling algorithm to measure similarity based on the underlying distribution of graph features. Furthermore, we employ the FlexSketch method to efficiently and rapidly generate compact and fixed-size feature representations to address the continuously growing dynamic graphs, thus maintaining graph similarity in a streaming environment.

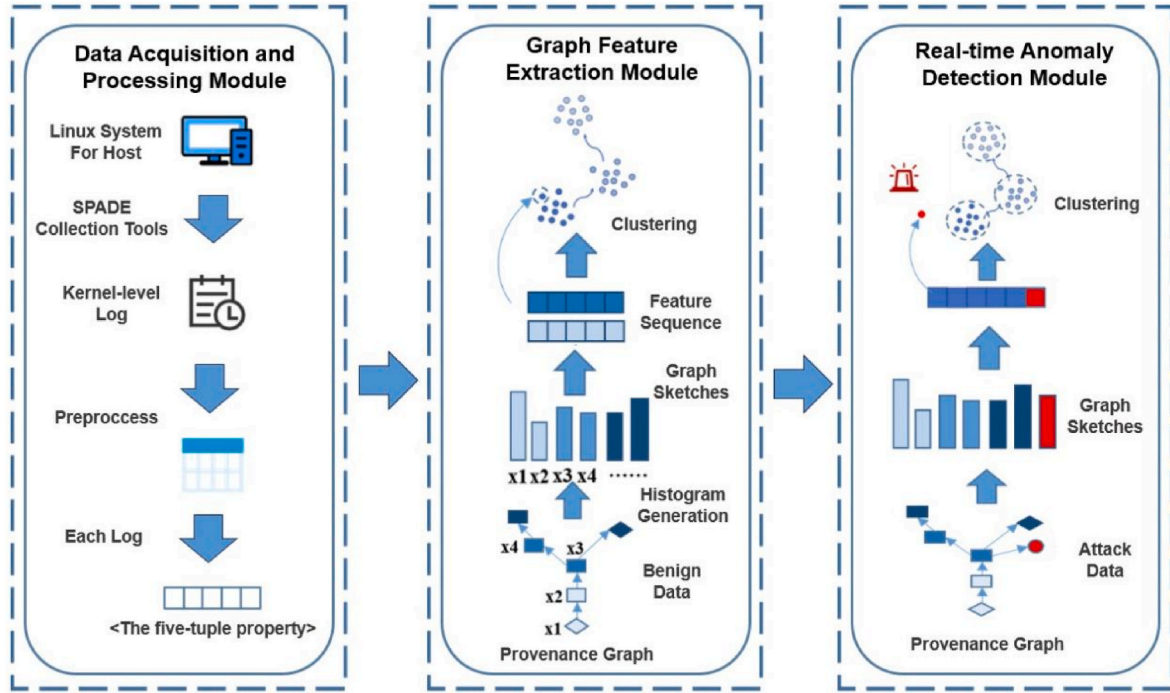


Fig. 1. The architecture of APT attack detection system based on provenance graph.

3. Systematic design

The proposed RT-APT consists of three modules, as shown in Fig. 1. (1) In the data acquisition and processing module, the SPADE (Indyk and Motwani, 1998) is a tool used for Linux system to collect the kernel-level logging data, and then Preprocess transforms the log data into the data structure. (2) In the graph feature extraction module, we introduce how to generate histograms using the WL subtree graph kernel algorithm which aggregates the contextual information of nodes. The histograms are incrementally updated when new edges arrive. Then a summary graph generation algorithm based on FlexSketch (Philbin and Zisserman, 2008) produces a fixed-size feature sequence (graph sketch) representing the evolving histogram. (3) In the real-time anomaly detection module, the K-means clustering anomaly detection algorithm is used to compare the summary graph of the current system with all trained normal models to determine if there is any abnormal behavior. Finally, we illustrate the working process through a real attack case.

3.1. Kernel-level log data collection and processing

The data collection tool needs to meet the following three requirements. (1) In APT scenarios, it is essential to ensure the security and integrity of information flow capture. APT attacks are insidious, and attackers often eliminate their traces in the system. Therefore, the collector must capture all system activities. (2) The collector must have long-term stability to resist persistent APT threats. (3) Data collection and transmission must be fast enough to achieve real-time detection. Therefore, in the data collection, we choose SPADE.

SPADE can collect two types of data structures: node-based and edge-based. For node-based data structures, it is necessary to cover attributes such as the type and properties of each node. The relationships (operations) between nodes are associated via the edge-based data structure. For edge-based data structures, a single edge represents an event. Edge-based data structures capture various relationships between nodes, enabling the association of two nodes that may be far apart. This is crucial for uncovering covert behaviors in APT attacks. During the construction of the provenance graph, entity nodes (subjects, objects) can be linked based on the edge-based data, and thus connections

between them can be established.

3.2. Graph feature extraction

3.2.1. Streaming histogram construction

In this paper, we take the WL subtree graph kernel algorithm as a graph embedding technique for the provenance graphs. This algorithm can rapidly and efficiently construct a histogram representing the entire history of system executions. The histogram is a counting statistical structure where each element corresponds to a node in the provenance graph. The value of each piece represents to what degree the complex contextual information aggregates, including causal relationships between objects, subjects, and activities, involving the corresponding node and all its neighboring nodes (Lü and Zhou, 2011). Furthermore, each element in the histogram describes a substructure of the graph and considers heterogeneous labels attached to nodes and edges and the temporal order of these edges.

The WL subtree graph kernel algorithm is a one-dimensional variant of the Weisfeiler-Lehman isomorphism test (Rajaraman and Ullman, 2011). The key idea of this algorithm is to expand node labels based on the ordered set of labels from neighboring nodes (e.g., those sorted according to the logical timestamps of neighboring nodes) and compress these expanded labels into new shortened labels. These two steps are repeated until the iteration reaches a predetermined maximum value n .

We briefly compare the W subtree kernel with other prominent kernels like the random walk kernel and shortestpath kernel. The random walk kernel, while useful for stochastic similarity measures, may overlook the deterministic causal structures inherent in kernel logs. The shortestpath kernel excels at identifying topological similarities but may not sufficiently distinguish the nuanced differences in the execution context of system activities. We conducted a series of benchmarks to compare the performance of these algorithms on graphs of different sizes. The results show that while all algorithms perform well on small-scale graphs, the WL subtree kernel algorithm still maintains high efficiency and accuracy on large-scale graphs. The WL subtree kernel's iterative label compression technique adeptly encapsulates the causal and contextual depth of provenance graphs making it an optimal choice for our purposes.

Using a designated labeling function, the WL subtree graph kernel algorithm first assigns specific labels to each node. Then, it calculates the frequency each label occurs in the graph, which forms a histogram vector. Next, a hash-based compression method is employed to transform the histogram vector into a fixed-size summary graph, which measures the similarity between different graphs. Finally, a clustering algorithm is applied to group the summary graphs, capturing behavioral changes during the execution of system and forming an evolutionary model. Fig. 2 illustrates these steps with an example. In step 1, the numbers 1–5 represent the number of labels assigned to each node. In each iteration, the labels from neighboring nodes are aggregated onto the current node, forming a set structure (step 2). However, the large number of neighboring nodes results in the increase of the length of the recent label, leading to insufficient storage space. Therefore, the expanded labels need to be compressed through a relabeling method, where a new label is generated by the labeling function (step 3). Finally, for a node, the relabeled label becomes the new label after one iteration (step 4).

The WL subtree graph kernel algorithm has a time complexity of $O(k*|E|)$, ensuring fast and efficient graph computation. Here, k represents the total number of iterations, and $|E|$ denotes the total number of edges. Specifically, in step 2, defining a multiset for all nodes requires an $O(|E|)$ operation. In step 3, sorting each multiset can be achieved by using counting sort (an implementation algorithm of bucket sort) since the range of multiset elements is limited, which is suitable for counting sort. The time complexity of this step is $O(|E|)$. Subsequently, compressing the expanded labels requires a time complexity of $O(|E|)$.

To further analyze the scalability of the algorithm, we conduct performance tests on graphs of different sizes to evaluate the efficiency of the algorithm when dealing with large scale datasets. We find that the computation time of the algorithm grows linearly as the graph size increases, which indicates its good scalability. However, on large-scale graphs, reasonable parameter selection and optimization become crucial in order to maintain system efficiency. For example, for graphs with millions of nodes and edges, we may need to adjust the number of iterations k to balance accuracy and computational cost.

This paper utilizes the WL subtree graph kernel algorithm to construct a histogram representation of the provenance graph. In the provenance graph, each node captures the surrounding graph structural information, including complex contextual information about causal relationships between objects, subjects, and activities (Lü and Zhou,

2011). Subsequently, these nodes are classified according to their expanded labels, which describe the R-hop neighborhood information of each node. These expanded node labels are constructed through iterative label propagation. For every node in the provenance graph, we take as input all node labels, incoming edge labels to each node, and source node labels for all the edges. For each node, a new label is then outputted, representing the aggregation of all input labels, and this process is repeated. The entire process is repeated R times to construct R -hop neighborhood labels. Accordingly, the provenance graph is represented as a histogram structure. If two graphs have similar distributions on similar label sets, they are considered equal.

3.2.2. Graph Sketches Generation method based on FlexSketch

Different from traditional histogram-based similarity (Sivic and Zisserman, 2003) (MacQueen et al., 1967) (Yadav and Rao, 2015), this paper proposes a graph sketch based method which uses FlexSketch (Philbin and Zisserman, 2008) to generate fixed-size vector representations of streaming histograms that are continuously updated and preserve the similarity between histograms. Measuring the similarity between streaming histograms is not easy because the number of histogram elements is unknown and constantly changes. Additionally, the underlying distribution of graph features is essential to measure the similarity of histograms rather than absolute counts. To this end, previous research has attempted to combine enumeration and manual feature engineering methods to pre-enumerate all possible histogram elements. However, given the potentially large number of nodes and edge labels, the resulting histogram data structure becomes sparse due to the multi-label provenance graph and a large number of neighborhood iterations.

This paper utilizes Locality Sensitive Hashing (LSH) (Jacob et al., 2008), a consistent weighted sampling-based approach to achieve. It leverages histogram normalization and generalized min-max (Jaccard) similarity to approximate histograms. Jaccard similarity (Kyrola et al., 2012) has been successfully applied to solve various real-world tasks based on machine learning, such as social network analysis (Friedman, 1997), text mining [54], and image processing [55]. The similarity between two graphs can be effectively computed by projecting their high-dimensional histogram representations into a lower-dimensional space. Specifically, the LSH method is a distribution over a family F of hash functions operating on a collection of objects. For two objects, m and n :

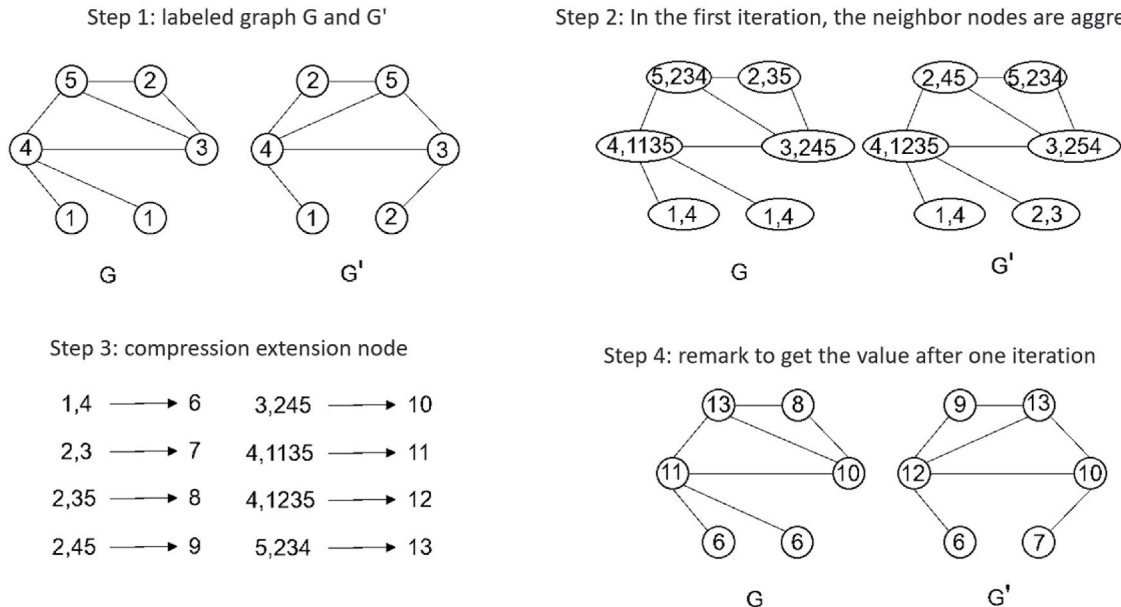


Fig. 2. Example of one-dimensional Weisfeiler-Lehman isomorphism test.

$$P_{h \in \mathcal{S}}[h(m) = h(n)] = \text{sim}(m, n) \quad (1)$$

where $h \in \epsilon$, h represents histogram elements, ϵ represents the set of histogram elements, and $\text{sim}(m, n)$ is a similarity function defined on the sets of objects. This paper adopts the normalized min-max similarity to measure the similarity between two histogram vectors, as shown in Equation (2), where a and b represent the units of the histograms.

$$\text{Sim}_{\min-\max}(H^a, H^b) = \frac{\sum_{h \in \epsilon} \min(H_h^a, H_h^b)}{\sum_{h \in \epsilon} \max(H_h^a, H_h^b)}, \sum_{h \in \epsilon} H_h^a = 1, \sum_{h \in \epsilon} H_h^b = 1 \quad (2)$$

Moreover, FlexSketch, generating graph sketches in this paper, can also address the concept drift. It processes streaming data dynamically, and decides when to forget the old data and constructs a new statistical model by measuring the difference between the current model and the recently sampled data. It can use minimal memory to estimate the probability density function (PDF) with a high speed and precision for both stationary (i.e., no concept drift) and non-stationary (i.e., concept drift) data streams.

Specifically, let S denote the data structure of the FlexSketch framework, which is composed of the latest data stream and statistical models of multiple versions denoted as M . It can be expressed as follows:

$$S = \{Q, M1, \dots, MNM, n1, \dots, nNM\} \quad (3)$$

The buffer Q represents the latest data set provided by the input data stream, and M_i denotes the i th histogram. We assume that the total number of histograms is greater than 2, that is, $NM \geq 2$, and n_i represents the number of data points used for updating M_i . A new histogram is constructed once significant changes are detected in the data stream. Therefore, the histograms within the data structure are created at different time points. According to that the convention $M1$ is the most recently added, if MNM is the oldest, indicating that M_{i+1} has undergone more updates than M_i , then $n_{i+1} > n_i$. Each histogram consists of a set of disjoint intervals called buckets, denoted as I_j , and each bucket has a frequency count m_j , where $j = 1, \dots, NB$. Therefore, M can be represented as:

$$M = \{I1, \dots, INb, m1, \dots, mNB\} \quad (4)$$

Assumed that FlexSketch can keep its stability and tolerate temporary linear outliers without changing the concept of data stream, it can achieve low computational overhead and high throughput, which is crucial for processing streaming data.

3.3. APT anomaly detection method based on clustering

Given graph sketches and a similarity measure, clustering is a commonly-utilized anomaly detection method to identify outliers. This paper leverages the capability of graph sketches to handle the streaming to create a model that captures normal variations in system behaviors. It is of paramount importance that the benign model is established during the training rather than the deployment process. In the deployment, building a dynamic model can lead to model poisoning after the prolonged APT attacks, significantly reducing accuracy.

A sequentially ordered feature sequence is created in the training to represent all benign behaviors generated within a complete system execution cycle. Then the K-means [56] algorithm clusters the graph sketches sequence, and the optimal value of K depends on the silhouette coefficient. The resulting K clusters represent the K elemental states of the system execution, such as the startup state, initialization state, stable state, and so on.

To ascertain the optimal number of clusters (K), we employed Silhouette Analysis, a statistical method that quantifies the similarity of an object to its own cluster compared to other clusters. This approach provides a measure of how close each point in one cluster is to the points in the neighboring clusters, with higher Silhouette coefficients indicating better defined clusters. We plotted the Silhouette coefficients for a

range of K values and identified the ‘elbow’ point where the rate of decrease sharply changes, suggesting an optimal K that balances the number of clusters with their cohesion and separation.

Although K-means is a basic clustering technique, Sakirin and Asif [57] have further explored the adaptability of K-means in dynamic environments and its potential to improve complex threat detection, and these insights can strengthen the theoretical foundation of our approach. Additionally, a sub-model is created for each training instance to capture the variations in the execution state during the runtime. The final model comprises sub-models from provenance graphs of all training data. By modeling the sub-models, the final model learns the changing behaviors at multiple time points. During the deployment detection, the anomaly detection follows the same streaming paradigm described in Subsection 3.2.1: periodically creating histograms and generating graph sketches. We compare a given graph sketch with all the sub-models learned in the training and then fit it into the clusters of each sub-model. Assuming monitoring starts when system starts up and tracks the system state transitions in each sub-model, a graph sketch must match with one of the current states or a state within a sub-model. Otherwise, it is considered an anomaly. Therefore, this paper can detect two types of abnormal behaviors: those that do not fit into existing clusters and those that represent invalid transitions between clusters.

There are diverse APT attack methods, and attackers often exploit zero-day vulnerabilities. It is difficult for the detection system to establish models for all attack behaviors. Therefore, this paper adopts a modeling approach based on normal behaviors. When two system states are similar, their corresponding feature vectors are similar. During the attack detection, we can calculate the distance between the feature vector of the current system state and the cluster center. Any behavior that exceeds a threshold distance determined by the training data of the model is considered an anomaly.

Furthermore, we explored alternative clustering algorithms to assess their comparative advantages over K-means. The DBSCAN algorithm [58], which does not require a predefined number of clusters and can handle noise in the data, was considered for its ability to identify outliers and discover clusters of arbitrary shapes. Hierarchical Clustering [59] was also examined for its ability to generate a dendrogram that visually represents the hierarchical structure of clusters. However, its computational complexity can be a limiting factor for large datasets, (Gaussian Mixture Models (GMM) [60]). offer a soft-clustering approach, allowing data points to belong to multiple clusters with certain probabilities, which can be beneficial in cases where clear cluster boundaries are not present. Nonetheless, the fitting process of GMMs can be computationally intensive and requires careful selection of the number of components.

Each clustering algorithm has its unique merits and is suited to different scenarios. The choice of the appropriate algorithm hinges on the characteristics of the data and the specific requirements of the analysis.

3.4. Case analysis

In this subsection, we illustrate the proposed method, RT-APT, using a real attack case. Given a provenance graph, rectangles represent processes, ellipses represent files, and diamonds represent networks.

Fig. 3 shows specific attack cases. This attack scenario is a Webshell attack from Apache. The victim host has port 80 open for web services (time points 1–3). In the web service, user-uploaded files are stored in var/www/html/uploads/shell.php. However, a file upload vulnerability exists, allowing users to upload any type of files and access it (time point 4). Consequently, an attacker scans for this vulnerability and uploads a Trojan file, establishing a connection and successfully obtaining a Webshell (time points 5–8). However, the acquired Webshell only has www-data user privileges, which are insufficient for the attacker’s need. Therefore, the attacker performs an elevation-of-privilege operation. During further infiltration, the attacker discovers that the administrator

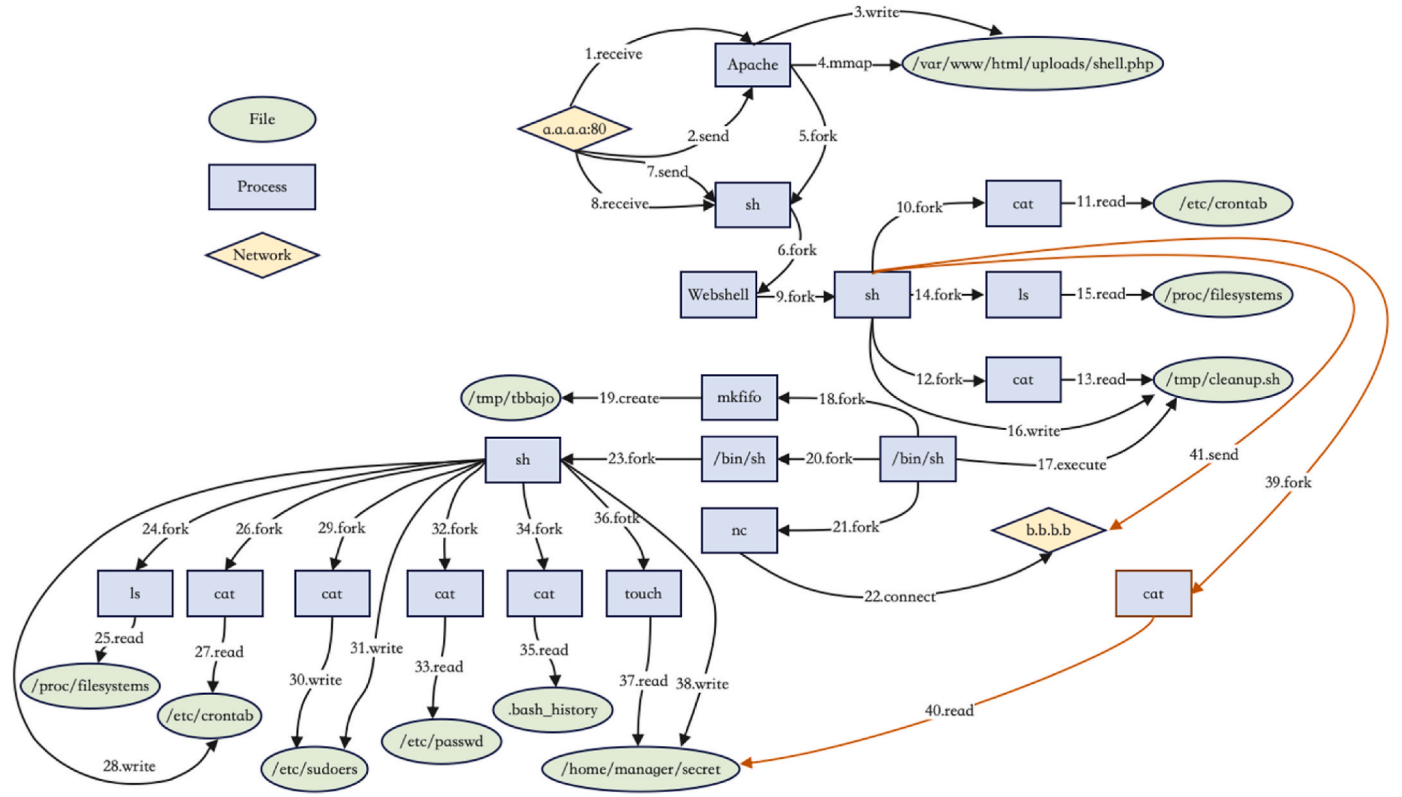


Fig. 3. The provenance graph of real attack cases.

periodically uses a cleanup.sh script to clear the folder where uploaded files are stored, as specified in the /etc/crontab file. By examining this script file, it is found that anyone has the permission to read and write. Thus, the attacker injects malicious code into this script file to gain reverse shell access (time points 9–16). Subsequently, the attacker begins to listen and wait for the administrator to execute the script file (time point 17), thereby acquiring administrator privileges. By running commands, the attacker identifies that the administrator belongs to the root group and possesses all the necessary permissions for the attacker's activities. Using the administrator's shell, the attacker can modify /etc/crontab and /etc/sudoers for persistent control (time points 24–38). Afterwards, the attacker can read valuable files on the host (time points 39–40) and send them to the organization's system via network transmission (time points 22, 41).

According to (Xiong et al., 2020), the ultimate goal of an APT attack is to steal sensitive information or cause damage. Therefore, during the attack, there are some common behavioral characteristics, such as executing sensitive commands, reading sensitive files, and transmitting sensitive data. These covert attack behaviors are rare in normal scenarios and can be learned and represented by the WL subtree graph kernel algorithm. Moreover, the emergence of these new operations (nodes) leads to changes in the underlying distribution of statistical features in the provenance graph learned from previous representations. To address the concept drift in streaming graphs, the FlexSketch graph sketching method is employed. Although this method reduces the influence of the old data on the current provenance graph, and no matter how far the event (behavior) occurred in time from the current system's execution state (time point 39), as long as it has a causal relationship with the most recently occurring operation (node) (e.g., the process "sh" associated with time point 39), the event and its neighborhood information will have an unreserved impact on the current features of provenance graph. Therefore, RT-APT can better differentiate between attack behaviors and normal behaviors, and separate anomalous provenance graphs from normal ones in the vector space.

4. Experiments and results

We conducted experiments to evaluate the accuracy and efficiency of RT-APT. Our evaluation focused on addressing the following research questions.

- (1) What should the number of hops (k-hop), sketch graph size (sketch-size), update triggering Interval (Interval), and decay rate (α) of the sketch graph be if our proposed detection system performs best?
- (2) How does our proposed detection system perform on the Lab, StreamSpot, and Unicorn datasets, respectively?
- (3) How is our system superior to StreamSpot and Unicorn?
- (4) What is the runtime performance of our proposed detection system?

4.1. Experimental datasets

The experimental data in this paper consists mainly of the Laboratory Dataset, StreamSpot Dataset, and Unicorn Dataset. Specifically, the optimal parameters were determined by analyzing the Laboratory Dataset. At the same time, the StreamSpot Dataset and Unicorn Dataset were used to compare RT-APT and existing state-of-the-art (SOTA) methods.

4.1.1. Laboratory dataset

In this paper, the laboratory dataset, primarily used to determine the optimal parameters for the experiments, can be grouped into an attack dataset and a benign dataset. Both of them were collected via the SPADE tool in a controlled laboratory environment. There are two teams involved in the experiments: defense team and attack team. The defense team was responsible for collecting audit data and detecting attacks from Linux platforms, including three hosts with Ubuntu 16.04

operating systems. The attack team was responsible for detecting vulnerabilities in the operating systems and carrying out attacks on the target systems. Additionally, the attack team produced benign background activities such as web browsing and document editing to simulate normal system behaviors.

- (1) Attack Dataset. We designed three APT attack scenarios, as shown in Table 1. APT datasets, T-1, T-2, and T-3 are generated by simulating real-world scenarios in the laboratory. Each group records all the activities performed by the system within a 24-h period.

To simulate APT attacks, the design of the attack path in the data collection followed a typical network kill chain model. This model consists of approximately seven stages, namely, reconnaissance (identifying the target and exploring its vulnerabilities), weaponization (designing backdoors and penetration plans), delivery (delivering the weapon), exploitation (triggering vulnerabilities on the victim's system), installation (installing backdoors or malicious software), command and control (C&C) (issuing remote commands to the victim), and actions on target (taking actions against the target) (Yadav and Rao, 2015).

To collect the data set T-1, we simulated a Webshell attack on the Apache web server. The details of T-1 have already been mentioned in Section 3.4, so we will not repeat them here.

To collect the data set T-2, we simulated a remote access Trojan attack on phishing websites. In this attack, the attacker leveraged the phishing website to entice the administrator to click a malicious link. Subsequently, the Trojan file was downloaded onto the victim's host. The attacker then monitored and waited for the administrator to execute the Trojan file, gaining ordinary user privileges. The subsequent privilege escalation was similar to that described in dataset T-1. Once privileges were elevated, the attacker modified sensitive files, establishing persistent control and exfiltrating valuable files and user information from the compromised system.

To collect the data set T-3, we simulate a fileless attack on a vulnerable program. In this attack, the attacker discovered a special service on port 29273 of the victim's host. This service had a vulnerability that the attackers exploited through string overflow to gain user privileges. Subsequently, the attacker crafted a script to trigger a string overflow, and eventually achieved the fileless attacks. As a result, the attacker obtained the user privileges associated with running the vulnerable service. The subsequent attack is the same with that in dataset T-1.

- (2) A benign dataset. This paper presents five benign scenarios as shown in Table 2. Benign datasets B-1, B-2, B-3, B-4, and B-5 are generated in a laboratory environment to simulate the real work of enterprise employees. Each dataset records all activities performed by the system within 24 h.

We regarded as benign scenarios normal system activities, such as writing files and scripts with the vim command, downloading datasets and resources (including the third-party libraries), upgrading software packages and so on. Since RT-APT is based on anomaly detection, it is

Table 1
Laboratory dataset features attack.

Dateset	Time (h)	Source Systems	Tools	Data Sources	Attack Descriptions
T-1	24	Ubuntu 16.04	SPADE	Laboratory	Apache Webshell attack
T-2	24	Ubuntu 16.04	SPADE	Laboratory	Phishing website Trojan attacks
T-3	24	Ubuntu 16.04	SPADE	Laboratory	Fileless attack

Table 2
Laboratory benign dataset characteristics.

Dateset	Time (h)	Source Systems	Tools	Data Sources	Attack Descriptions
B-1	24	Ubuntu 16.04	SPADE	Laboratory	no data collection operations
B-2	24	Ubuntu 16.04	SPADE	Laboratory	wget downloads, software upgrades
B-3	24	Ubuntu 16.04	SPADE	Laboratory	service startups
B-4	24	Ubuntu 16.04	SPADE	Laboratory	file operations, script writing
B-5	24	Ubuntu 16.04	SPADE	Laboratory	website browsing, chatting, or email sending

necessary to collect benign scenario data as many as possible to avoid false positive alerts generated due to system behaviors that do not perform in training.

Table 3 shows the statistical results obtained after we preprocessed all the obtained benign and attack data. The first column of the table represents the dataset label, the second column represents the number of provenance graphs in the dataset, the third column represents the average number of nodes in the graph, the fourth column represents the average number of edges, and the last column represents the current dataset size.

4.1.2. StreamSpot dataset

The StreamSpot dataset, collected by the SystemTap kernel tool (Jacob et al., 2008), is publicly available only in preprocessed format. As shown in Table 4, it consists of one drive-by download attack scenario and five benign host activity scenarios, i.e., watching YouTube, browsing CNN, downloading files, viewing emails, and playing games. Each scenario was run 100 times, resulting in 100 graphs. The attack scenario involves downloading files from a malicious URL, exploiting Flash vulnerabilities, and gaining root access to the host. Table 4 provides a summary of this dataset. By analyzing these dataset, we notice that each benign graph in the dataset has about 173K edges on average, while the attack graph has about 28K edges on average (Han et al., 2020).

4.1.3. Unicorn Dataset

Unicorn is a machine learning based attack anomaly detection algorithm on provenance graphs, and capable of automatically constructing system behavior models. Firstly, the provenance graph, which describes system behaviors, is transformed into a sequence of feature vectors. The feature vectors are then clustered to generate system states. Subsequently, the changes of system states are described with automaton models. An automaton model is established for each normal sequence, and during testing, the current state is examined for detection.

Unicorn is designed with two APT attack scenarios and benign scenarios as shown in Table 5, and there is a certain level of similarity between the benign and attack scenarios. The two attack scenarios and the benign scenario are analyzed separately, and the results are shown in Table 5. Unicorn simulated two APT supply chain attacks (SC-1 and SC-

Table 3
Characteristics of the Laboratory dataset.

Dateset	Graphs	Avg. nodes	Avg. edge	Dateset Size(GiB)
T-1	20	132457	1068953	15.57
T-2	20	162587	1141039	16.69
T-3	20	86785	958924	13.87
B-1	100	8647	41228	3.49
B-2	100	87553	451863	31.01
B-3	100	61867	369252	29.21
B-4	100	17300	104260	7.79
B-5	100	10758	62543	5.47

Table 4

Characteristics of the StreamSpot dataset.

Scenario	Graphs	Avg. nodes	Avg. edge	Dateset Size(GiB)
YouTube	100	8292	113229	0.3
Gmail	100	6827	37382	0.1
Downloading	100	8831	310814	1
VGame	100	8637	112958	0.4
CNN	100	8990	294903	0.9
Attack	100	8891	28423	0.1

Table 5

Characteristics of the Unicorn dataset.

Dateset	Label	Graphs	Avg. nodes	Avg. edge	Dateset Size(GiB)
SC-1	Benign	125	265424	975226	64
	Attack	25	257156	957968	12
SC-2	Benign	125	238338	911153	59
	Attack	25	243658	949887	12

2) using a Continuous Integration (CI) approach. It utilized CamFlow to capture the provenance of the entire system, including background activities.

4.2. Experimental results and analysis

4.2.1. Experimental environment

The experiments in this paper were conducted on a server running Ubuntu 16.04. The server is equipped with an Intel Xeon E5-2680 v4 CPU with 14 cores clocked at 2.4 GHz, 128 GB of RAM, and four NVIDIA GeForce RTX 2080Ti GPUs. The real-time detection algorithm proposed in this paper is implemented using C++ and Python. We analyzed the system log data of approximately 150 GB. GraphChi (Kyrola et al., 2012) was employed to make efficient graph computation.

4.2.2. Experimental evaluation index

RT-APT takes a binary classification for benign and attack samples. Therefore, in this paper, we have four metrics: TP (true positive), representing the number of attack samples correctly classified; FP (false positive), representing the number of attack samples mistakenly classified; TN (true negative), representing the number of benign samples correctly classified; and FN (false negative), representing the number of benign samples mistakenly classified.

Accordingly, to classify the samples, we get the following evaluation metrics:

$$\text{precision} = \frac{TP}{TP + FP} \quad (5)$$

$$\text{recall} = \frac{TP}{TP + FN} \quad (6)$$

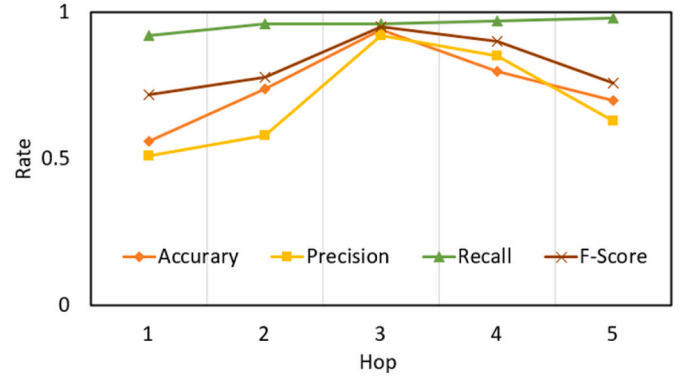
$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

$$F\text{-score} = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (8)$$

4.2.3. Parameter configuration

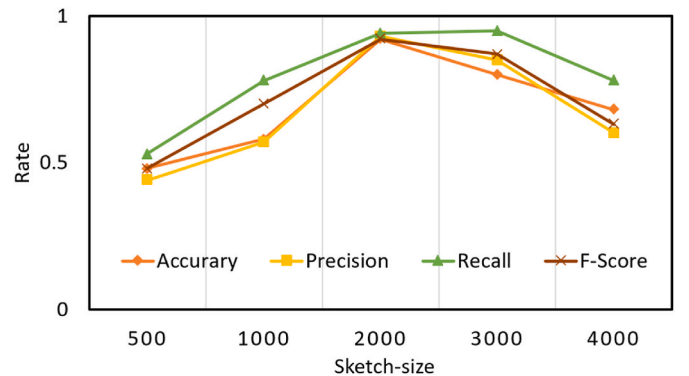
In this paper, we use four important parameters from which we can find how our method performs and when it performs best.

- (1) The parameter "k-hop" in the provenance graph denotes the neighborhood size of each node. A larger k-hop indicates that the current model captures a larger size of neighborhood, thereby incorporating more contextual information. The influence of the k-hop size on the detection is illustrated in Fig. 4. We can find that when k-hop is set to 3, all the precision, recall, and accuracy are

**Fig. 4.** The influence of parameter k-hop.

the highest, implying the best detection performance. However, when k-hop exceeds 3, the captured contextual information may become irrelevant and unintentionally mask potential attacks, leading to a decline in detection effectiveness.

- (2) The parameter "sketch-size" represents the size of the summary graph obtained by taking the growing histogram as a fixed-size sketch. A larger sketch-size allows the graph sketches to contain more information about the streaming updates of the histogram, thereby reducing errors in normalizing the maximum and minimum similarity and improving the detection precision, recall, and accuracy. The influence of the sketch-size is shown in Fig. 5. We can find that our method performs best when the sketch-size is set to 2000. When the value of sketch-size exceeds 2000, it is influenced by the dimensional factors in the clustering detection (Friedman, 1997), leading to a decline in detection performance.
- (3) "Interval of Sketch Generation" represents the condition for triggering updates of the provenance graph in a streaming environment. Typically, the Interval value can indicate the number of newly added edges or nodes. In this paper, we first make the Interval represent the number of edges. The influence of the Interval size is shown in Fig. 6. A smaller Interval makes the adjacent provenance graphs appear more similar, resulting in a higher false negative count and lower recall and accuracy. Conversely, when the Interval is too large, the granularity of changes between the provenance graphs would become great, making it difficult to highlight the attack behaviors and thus causing the adjacent graphs to appear similar as well. As shown in Fig. 6, the Interval value should be set to 3000. Similarly, if we make the Interval represent the number of nodes, the optimal Interval* value should be set to 500 as shown in Fig. 7.
- (4) The decay rate (α) represents the gradual forgetting of old data in the provenance graph with a decay rate of α . The gradual forgetting method helps the detection system to focus on the

**Fig. 5.** The influence of parameter sketch-size.

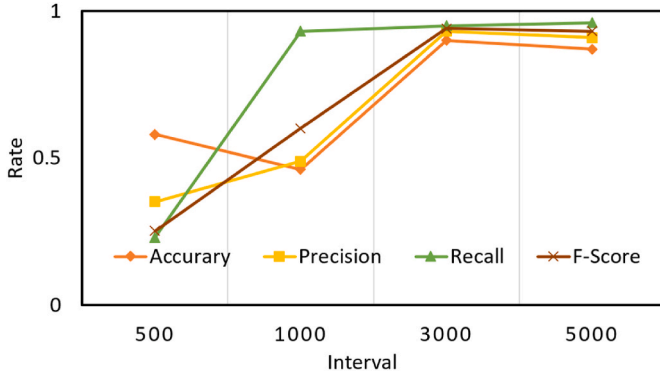


Fig. 6. The influence of parameter Interval.

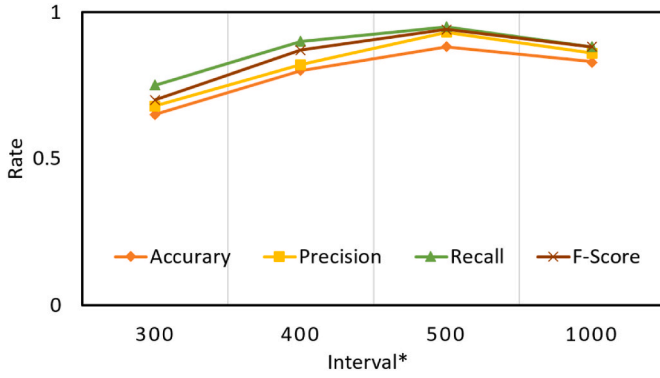


Fig. 7. The influence of parameter Interval*.

current state of system execution (i.e., the most recent part of the provenance graph) and any part of the provenance graph causally related to the current execution, while preserving the log. The influence of different α is shown in Fig. 8. We can find that when α is set to 0.03, the graph sketches achieve a good balance between the historical histogram and the current histogram.

4.2.4. Experimental results

In this paper, according to the silhouette coefficient we set the optimal number of clusters as $K = 6$. All $500 \times 0.8 = 400$ benign graphs are used for training. The remaining $500 \times 0.2 = 100$ benign graphs and 60 attack graphs are used for testing. In the experiments, the optimal parameters are set as: $k\text{-hop} = 3$, $\text{sketch-size} = 2000$, $\text{Interval} = 3000$, and $\alpha = 0.03$. We use the 5-fold cross-validation in the experiments.

Since RT-APT models normal behaviors, we plot a precision-recall (PR) curve as shown in Fig. 9 to determine the optimal threshold for the testing. The area between the PR curve and the coordinate axis is

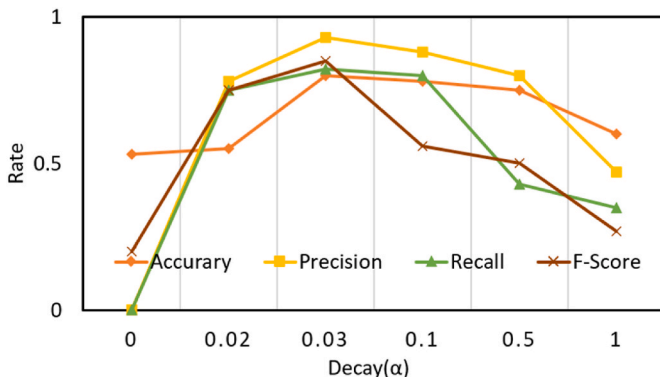
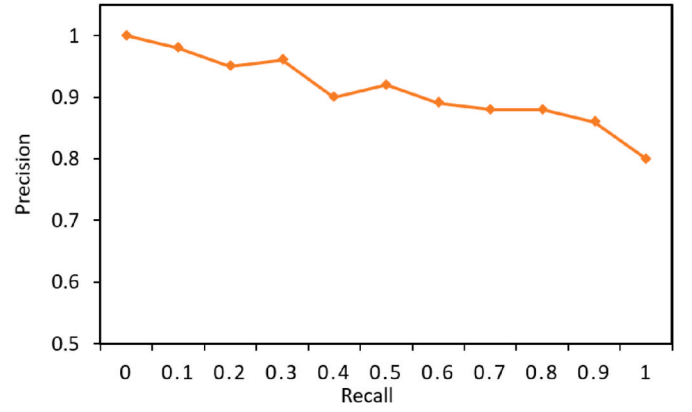
Fig. 8. The influence of parameter α .

Fig. 9. The PR curve of experimental results.

used as an evaluation metric, where a larger area indicates a better overall classification across different threshold values.

4.2.5. Precision experiment

This paper compares our proposed RT-APT with existing detectors on three datasets: the laboratory dataset, the StreamSpot dataset, and the Unicorn dataset.

On the laboratory dataset, this paper conducts experiments by using 80% of the total 500 benign graphs for training. The remaining 20% of the benign graphs combined with 60 attack graphs are for testing. In the experiments, the proposed method is configured with optimal parameters: $k\text{-hop} = 3$, $\text{sketch-size} = 2000$, and $\text{Interval} = 3000$.

The experimental results are shown in Table 6, and our proposed RT-APT method based on the WL subtree kernel algorithm outperforms the existing detection methods in terms of precision, recall, and F-score.

On the StreamSpot dataset, this paper conducts experiments by using 80% of the total 500 benign graphs for training. The remaining 20% of the benign graphs combined with 100 attack graphs are for testing. Similarly, the proposed method is configured with optimal parameters: $k\text{-hop} = 3$, $\text{sketch-size} = 2000$, and $\text{Interval} = 3000$.

The results on the StreamSpot dataset shown in Table 7 indicate that the proposed method is more suitable for APT attack detection on the StreamSpot dataset. This is because it incorporates multi-hop graph exploration when aggregating neighboring node information and utilizes an automaton modeling strategy to model normal behaviors.

Then we compare our evaluation metrics with the Unicorn (Han et al., 2020) detector on the Unicorn dataset. The experiments involve randomly dividing 125 benign graphs into 5 groups for 5-fold cross-validation. Specifically, each training set consists of 100 benign graphs. After modeling normal behaviors, the false positive of the remaining 25 benign graphs is evaluated. Then, the model is evaluated in false negatives on 25 attack graphs. This process is repeated for each group, and the average results are recorded. The results on the Unicorn dataset presented in Table 8 indicate that compared with the Unicorn method, the proposed method is better at distinguishing normal and abnormal behavior in long-term attack scenarios.

In summary, RT-APT exhibits superior performance when evaluated on the laboratory dataset, StreamSpot dataset, and Unicorn dataset, as depicted in Fig. 10.

The primary reason why we choose the FlexSketch algorithm is that

Table 6
Comparison of different detection methods on the Lab dataset.

Detector	precision	recall	accuracy	F-score
StreamSpot (baseline)	0.84	0.72	0.86	0.82
Unicorn	0.95	0.92	0.97	0.94
RT-APT	0.98	0.96	0.94	0.97

Table 7

Comparison of different detection methods on the StreamSpot dataset.

Detector	precision	recall	accuracy	F-score
StreamSpot (baseline)	0.74	0.72	0.66	0.78
Unicorn	0.98	0.93	0.96	0.94
RT-APT	0.96	0.94	0.95	0.95

Table 8

Comparison of different detection methods on the Unicorn dataset.

Unicorn Dataset	Detector	precision	recall	accuracy	F-score
SC-1[13](baseline)	StreamSpot	0.80	0.88	0.85	0.85
	Unicorn	0.85	0.96	0.90	0.90
	RT-APT	0.88	0.92	0.90	0.90
SC-2[13] (baseline)	StreamSpot	0.72	0.72	0.70	0.74
	Unicorn	0.75	0.80	0.77	0.78
	RT-APT	0.84	0.88	0.86	0.85

it can capture features in long sequences more effectively. In real-world network attack scenarios, APT attack cycles are long, and the corresponding feature sequences are lengthy. The proposed method can extract patterns from long sequences more efficiently, and adapt to the characteristics of APT attacks in real scenarios. Although the Unicorn algorithm also utilizes feature sequences, it tends to overlook the subtle differences of the anomalies and normal behaviors in matching state machine templates. This study employs the FlexSketch algorithm, which constructs a new statistical model by measuring the differences between the current model and the most recent sampled data. While capturing sequence features, the algorithm integrates historical and current features of the feature sequences. The resulting sequence feature calculation incorporates the entire sequence's characteristics, preserving and accumulating minor differences. This increases the distance between normal and abnormal behaviors in the feature space.

The FlexSketch algorithm adeptly manages high-dimensional data by utilizing Local Sensitive Hashing (LSH) techniques, which efficiently approximate the similarity of feature vectors in high-dimensional spaces and maintain the integrity of temporal relationships. This is achieved by dynamically processing data streams, deciding the optimal time to phase out outdated information, and constructing new statistical models that consider both historical and recent data points. The algorithm's consistent weighted sampling approach ensures stability and adaptability to concept drift, while the decay rate parameter (α) facilitates a balanced focus on the current system execution state and its causal dependencies as recorded in the provenance graph. This harmonized approach allows FlexSketch to effectively capture and analyze the evolving patterns within the data, without being overwhelmed by the

complexities introduced by high dimensionality or temporal dynamics.

To assert the robustness and scalability of the FlexSketch algorithm, we conducted additional experiments to evaluate its performance under varied environments and different load conditions. The FlexSketch algorithm was tested with varying data set sizes and streaming rates to simulate diverse real-world scenarios.

Our results indicate that FlexSketch maintains a consistent performance profile across a wide range of conditions. Even under high-load situations, the algorithm demonstrates minimal memory overhead and acceptable processing latency, outperforming other state-of-the-art algorithms in terms of scalability.

4.3. Overhead

In this subsection, we analyze the runtime overhead, including runtime performance, and memory consumption and CPU utilization.

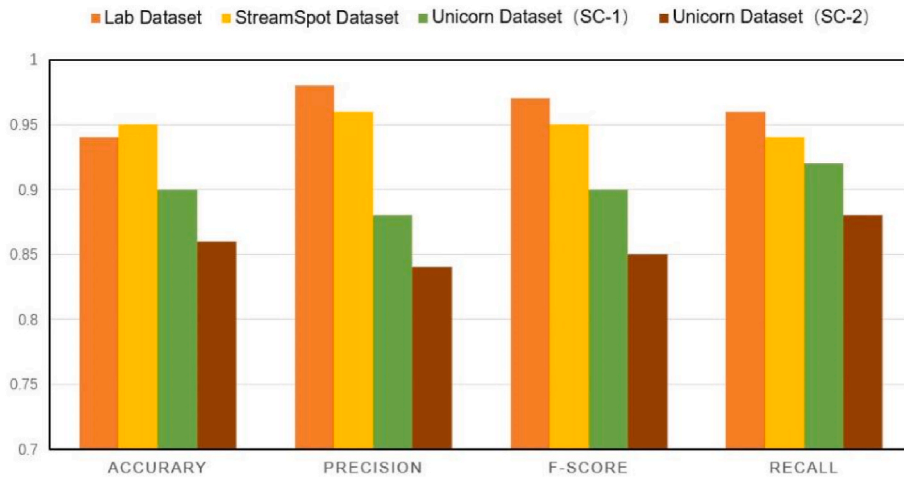
4.3.1. Runtime performance

In APT scenarios, runtime performance is crucial as intrusion detection systems (IDS) which often need to monitor systems in real time. In this subsection, to grasp the runtime performance of RT-APT, we take the same parameter settings as in Subsection 4.2.3. We conduct experiments on a laboratory dataset. The results demonstrate the trade-off between accuracy and runtime performance. Table 9 records the average number of edges processed within a given time (data processing rate) under different parameter settings. This quantifies the speed at which our method processes data. In Table 9, "inline" represents the speed close to that of the SPADE collection system (data generation rate). Smaller differences between the detection processing speed and the inline rate indicate better runtime performance. The difference in Table 9 is obtained by calculating the average processing edge speed under different parameters minus the difference of processing edge

Table 9

The impact of different parameter Settings on running performance.

Parameter	Value	Average processing speed (bars/sec)	Difference%
inline (baseline)	N/A	267.12	0
k-hop	2	265.25	-0.7
	3	263.38	-1.4
	4	262.84	-1.6
sketch-size	1000	266.90	-0.8
	2000	266.82	-1.1
	3000	252.30	-5.6
Interval	2000	267.06	-0.2
	3000	266.31	-0.3
	4000	266.31	-0.3

**Fig. 10.** Metric comparison of the proposed method in three datasets.

speed under the inline condition, and then dividing with the processing edge speed under the inline condition.

In general, RT-APT is relatively insensitive to variations in these parameters at runtime. Under the optimal parameter settings (highlighted in bold in the table), the differences with the baselines are around 1%. This means that the proposed method can perform real-time detection effectively.

4.3.2. Memory overhead and CPU usage

In this subsection, we test the memory consumption and CPU utilization. The results are shown in Table 10. It can be observed that the proposed system has a memory consumption of around 600 MB on various datasets. Compared with the Unicorn method (with a memory consumption of 687 MB under optimal detection performance), our proposed system reduces the memory consumption by approximately 15%. In comparison to the StreamSpot method, the average memory consumption is reduced by about 23%. The average CPU utilization remains stable at around 17.6%, indicating a clear advantage over StreamSpot and Unicorn methods.

5. Discussion

The RT-APT system demonstrates a robust generalization capability, vital for its effectiveness across various APT detection scenarios. It exhibits consistent performance across multiple datasets, such as Laboratory, StreamSpot, and Unicorn, showcasing its adaptability in different operational environments. This cross-dataset generalization enables RT-APT to maintain high detection accuracy in realworld applications. Moreover, in terms of identifying novel and unknown attacks, the RT-APT system is robust because it analyzes behaviors instead of relying on specific attack signatures.

Despite these strengths, RT-APT has certain limitations. It assumes a secure initial modeling phase to capture normal system behaviors, which is crucial for distinguishing benign from malicious activities. However, this assumption may occasionally lead to false positives when encountering new or unknown attack patterns. The effectiveness of the system also depends on the integrity of the log data, and the complexity of APTs brings the risk of data leakage, which may affect the detection effect.

While the FlexSketch algorithm excels in handling high-dimensional data and time series, it does have some limitations. For instance, when dealing with data that exhibit very complex patterns, the algorithm may require more computational resources to maintain high accuracy. Furthermore, the performance of the algorithm may be affected by the choice of hash functions and parameter settings, necessitating careful adjustment during actual deployment.

Parameter tuning presents another challenge, as RT-APT requires precise configuration of parameters like k-hop, sketch-size, and update intervals to optimize performance. This can be complex across different systems and conditions. Additionally, the reliance on external tools like SPADE for data collection introduces potential reliability issues.

While RT-APT marks a step forward in APT detection, ongoing research and development are necessary to overcome these limitations and refine the system's capabilities.

6. Conclusion

This paper proposes a real-time method for detecting APT attacks based on provenance graphs constructed from log data. The proposed method uses graph embedding techniques to consider rich contextual information and long-term causal relationships within the provenance

Table 10

Average memory overhead and average CPU usage.

	Detector	Lab Dataset	StreamSpot Dataset	Unicorn Dataset
Average memory cost (MB)	RT-APT	597	588	609
	StreamSpot	734	726	765
	Unicorn	686	664	586
Average CPU usage (%)	RT-APT	18.6	17.4	16.9
	StreamSpot	23	18	25.4
	Unicorn	19.2	19.6	14.3

graph. This allows for extensive and efficient graph analysis computations while enabling rapid updates of feature vectors in streaming environments. Experimental evaluations conducted on the StreamSpot dataset, Unicorn dataset, and in-house dataset demonstrate that the proposed method outperforms the state-of-the-art (SOTA) approaches.

CRediT authorship contribution statement

Zhengqiu Weng: Writing – review & editing, Writing – original draft, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Weinuo Zhang:** Software, Formal analysis, Data curation. **Tiantian Zhu:** Methodology, Investigation, Funding acquisition, Conceptualization. **Zhenhao Dou:** Validation, Software, Resources, Data curation. **Haofei Sun:** Writing – original draft, Visualization, Software. **Zhanxiang Ye:** Writing – original draft, Supervision, Project administration, Funding acquisition. **Ye Tian:** Writing – review & editing, Visualization, Validation, Software.

Data availability

The data used in this study are sourced from three distinct datasets. The Unicorn dataset can be accessed from <https://drive.google.com/drive/folders/1QlbUFWAGq3Hpl8wVdzOdIoZLFxkII4EK>. The StreamSpot dataset can be accessed and downloaded from <https://sbustreamspot.github.io/>. Interested parties may request our Laboratory dataset by contacting the authors via email at derisweng@wzut.edu.cn. All of these datasets are publicly available and can be obtained without restrictions.

Funding

This research was funded by Wenzhou Cyber Security Detection and Protection Technology Research Center (No. WZKF2023001 and No. WZKF2023002), supported by Wenzhou Major Scientific and Technological Innovation Research Project (No. ZG2023031 and No. ZG2024007), funded by Wenzhou Basic scientific Research Project (No. G2024033), supported by National Natural Science Foundation of China (No. 62273263), supported by Fundamental Research Funds for the Provincial Universities of Zhejiang (No. RF-A2023009).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

The author is an Editorial Board Member/Editor-in-Chief/Associate Editor/Guest Editor for *Journal of Network and Computer Applications* and was not involved in the editorial review or the decision to publish this article.

Data availability

Data will be made available on request.

References

- Akoglu, L., Tong, H., Koutra, D., 2015. Graph based anomaly detection and description: a survey. *Data Min. Knowl. Discov.* 29, 626–688.
- Alshamrani, A., Myneni, S., Chowdhary, A., Huang, D., 2019. A survey on advanced persistent threats: techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys & Tutorials* 21 (2), 1851–1877.
- Baker, L.D., McCallum, A.K., 1998. Distributional clustering of words for text classification. In: *The 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 96–103.
- Berlingerio, M., Koutra, D., Eliassi-Rad, T., Faloutsos, C., 2012. A Scalable Approach to Size-independent Network Similarity. *ArXiv Preprint ArXiv: 12092684*.
- Berrada, G., Cheney, J., Benabderahmane, S., Maxwell, W., Mookherjee, H., Theriault, A., Wright, R., 2020. A baseline for unsupervised advanced persistent threat detection in system-level provenance. *Future Generat. Comput. Syst.* 108, 401–413.
- Chapelle, O., Haffner, P., Vapnik, N.V., 1999. Support vector machines for histogram-based image classification. *IEEE Trans. Neural Network.* 10 (5), 1055–1064.
- Chen, T., Dong, C., Lv, M., Song, Q., Liu, H., Zhu, T., Xu, K., Chen, L., Ji, S., Fan, Y., 2022. Apt-kg: an intelligent apt detection system based on threat knowledge and heterogeneous provenance graph learning. *IEEE Trans. Dependable Secure Comput.* <https://doi.org/10.1109/TDSC.2022.3229472> (Access, Dec. 26, 2022).
- Ding, Q., Katenka, N., Barford, P., Kolaczek, E., Crovella, M., 2012. Intrusion as (anti) social communication: characterization and detection. In: *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 886–894.
- Feng, H.H., Kolesnikov, O.M., Fogla, P., Lee, W., Gong, W., 2003. Anomaly detection using call stack information. In: *2003 Symposium on Security and Privacy*, 2003. IEEE, pp. 62–75.
- Friedman, J.H., 1997. On bias, variance, 0/1—loss, and the curse-of-dimensionality. *Data Min. Knowl. Discov.* 1, 55–77.
- Gao, J., Liang, F., Fan, W., Wang, C., Sun, Y., Han, J., 2010. On community outliers and their efficient detection in information networks. In: *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 813–822.
- Ghani, A., Tariq, D., 2012. Spade: support for provenance auditing in distributed environments. In: *ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing*. Springer, pp. 101–120.
- Han, X., Pasquier, T., Bates, A., Mickens, J., Seltzer, M., 2020. Unicorn: runtime provenance-based detector for advanced persistent threats. In: *27TH ANNUAL NETWORK AND DISTRIBUTED SYSTEM SECURITY SYMPOSIUM (NDSS 2020)*.
- Hassan, W.U., Guo, S., Li, D., Chen, Z., Jee, K., Li, Z., Bates, A., 2019. Nodoe: Combating threat alert fatigue with automated provenance triage. In: *Network and Distributed Systems Security Symposium*.
- Hassan, W.U., Bates, A., Marino, D., 2020. Tactical provenance analysis for endpoint detection and response systems. In: *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 1172–1189.
- Hodge, V., Austin, J., 2004. A survey of outlier detection methodologies. *Artif. Intell. Rev.* 22, 85–126.
- Hossain, M.N., Milajerdi, S.M., Wang, J., Eshete, B., Gjomemo, R., Sekar, R., Stoller, S., Venkatakrishnan, V., 2017. {SLEUTH}: Realtime attack scenario reconstruction from {COTS} audit data. In: *26th USENIX Security Symposium (USENIX Security 17)*, pp. 487–504.
- Hossain, M.N., Sheikh, S., Sekar, R., 2020. Combating dependence explosion in forensic analysis using alternative tag propagation semantics. In: *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 1139–1155.
- Hunt, P., Konar, M., Junqueira, F.P., Reed, B., 2010. Zookeeper: wait-free coordination for internet-scale systems. In: *2010 USENIX Annual Technical Conference (USENIX ATC 10)*.
- Indyk, P., Motwani, R., 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In: *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, pp. 604–613.
- Jacob, B., Larson, P., Leitao, B., Da Silva, S., 2008. Systemtap: instrumenting the linux kernel for analyzing performance and functional problems. *IBM Redbook* 116.
- Kang, W., Son, B., Heo, K., 2022. Tracer: signature-based static analysis for detecting recurring vulnerabilities. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1695–1708.
- Kapoor, M., Melton, J., Ridenhour, M., Krishnan, S., Moyer, T., 2021. Prov-gem: automated provenance analysis framework using graph embeddings. In: *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, pp. 1720–1727.
- Kyrola, A., Blelloch, G., Guestrin, C., 2012. {GraphChi}: {Large-Scale} graph computation on just a {PC}. In: *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*, pp. 31–46.
- Liu, C., Yan, X., Yu, H., Han, J., Yu, P.S., 2005. Mining behavior graphs for “backtrace” of noncrashing bugs. In: *Proceedings of the 2005 SIAM International Conference on Data Mining*. SIAM, pp. 286–297.
- L’u, L., Zhou, T., 2011. Link prediction in complex networks: a survey. *Phys. Stat. Mech. Appl.* 390 (6), 1150–1170.
- MacQueen, J., et al., 1967. Some methods for classification and analysis of multivariate observations. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297. Oakland, CA, USA.
- Maggi, F., Matteucci, M., Zanero, S., 2008. Detecting intrusions through system call sequence and argument analysis. *IEEE Trans. Dependable Secure Comput.* 7 (4), 381–395.
- Manzoor, E., Milajerdi, S.M., Akoglu, L., 2016. Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1035–1044.
- Milajerdi, S.M., Gjomemo, R., Eshete, B., Sekar, R., Venkatakrishnan, V., 2019a. Holmes: real-time apt detection through correlation of suspicious information flows. In: *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 1137–1152.
- Milajerdi, S.M., Eshete, B., Gjomemo, R., Venkatakrishnan, V., 2019b. Poirot: aligning attack behavior with kernel audit records for cyber threat hunting. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1795–1812.
- Mutz, D., Robertson, W., Vigna, G., Kemmerer, R., 2007. Exploiting execution context for the detection of anomalous system calls. In: *Recent Advances in Intrusion Detection: 10th International Symposium, RAID 2007, Gold Coast, Australia, September 5–7, 2007. Proceedings 10*. Springer, pp. 1–20.
- Pan, X., Cao, Y., Du, X., He, B., Fang, G., Shao, R., Chen, Y., 2018. Flowcog: context-aware semantics extraction and analysis of information flow leaks in android apps. In: *27th USENIX Security Symposium (USENIX Security 18)*, pp. 1669–1685.
- Papadimitriou, P., Dasdan, A., Garcia-Molina, H., 2010. Web graph similarity for anomaly detection. *Journal of Internet Services and Applications* 1, 19–30.
- Park, N., Kim, S., 2021. Flexsketch: estimation of probability density for stationary and nonstationary data streams. *Sensors* 21 (4), 1080.
- Perozzi, B., Akoglu, L., Iglesias Sánchez, P., Müller, E., 2014. Focused clustering and outlier detection in large attributed graphs. In: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1346–1355.
- Philbin, J., Zisserman, A., 2008. Near duplicate image detection: min-hash and tf-idf weighting. In: *Proceedings of the British Machine Vision Conference*, vol. 3, p. 4.
- Piskozub, M., De Gaspari, F., Barr-Smith, F., Mancini, L., Martinovic, I., 2021. Malphase: fine-grained malware detection using network flow data. In: *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, pp. 774–786.
- Qu, Z., Guo, G., Shao, Z., Rastogi, V., Chen, Y., Chen, H., Hong, W., 2017. Appshield: enabling multi-entity access control cross platforms for mobile app management. In: *Security and Privacy in Communication Networks: 12th International Conference, SecureComm 2016, Guangzhou, China, October 10–12, 2016, Proceedings 12*. Springer, pp. 3–23.
- Rajaraman, A., Ullman, J.D., 2011. Mining of Massive Datasets. Cambridge University Press.
- Shervashidze, N., Schweitzer, P., Van Leeuwen, E.J., Mehlhorn, K., Borgwardt, K.M., 2011. Weisfeiler-lehman graph kernels. *J. Mach. Learn. Res.* 12 (9).
- Shi, L., Ming, J., Fu, J., Peng, G., Xu, D., Gao, K., Pan, X., 2020. Vahunt: warding off new repackaged android malware in appvirtualization’s clothing. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp. 535–549.
- Sivic and Zisserman, 2003. Video Google: A Text Retrieval Approach to Object Matching in Videos. IEEE.
- Song, W., Ming, J., Jiang, L., Xiang, Y., Pan, X., Fu, J., Peng, G., 2021. Towards transparent and stealthy android os sandboxing via customizable container-based virtualization. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2858–2874.
- Vishwanathan, S.V.N., Schraudolph, N.N., Kondor, R., Borgwardt, K.M., 2010. Graph kernels. *J. Mach. Learn. Res.* 11, 1201–1242.
- Weisfeiler, B.Y., Leman, A.A., 1968. A reduction of a graph to a canonical form and an algebra arising during this reduction. *Nauchno-Tekhnicheskaya Informatsiya* 2 (9), 12–16.
- Xiong, C., Zhu, T., Dong, W., Ruan, L., Yang, R., Cheng, Y., Chen, Y., Cheng, S., Chen, X., 2020. Conan: a practical real-time apt detection system with high accuracy and efficiency. *IEEE Trans. Dependable Secure Comput.* 19 (1), 551–565.
- Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K.-i., Jegelka, S., 2018. Representation learning on graphs with jumping knowledge networks. In: *International Conference on Machine Learning*. PMLR, pp. 5453–5462.
- Yadav, T., Rao, A.M., 2015. Technical aspects of cyber kill chain. In: *Security in Computing and Communications: Third International Symposium, SSCC 2015, Kochi, India, August 10–13, 2015. Proceedings 3*. Springer, pp. 438–452.
- Yu, L., Luo, B., Ma, J., Zhou, Z., Liu, Q., 2020. You are what you broadcast: identification of mobile and iot devices from (public) wifi. In: *29th USENIX Security Symposium (USENIX Security 20)*, pp. 55–72.
- Zengy, J., Wang, X., Liu, J., Chen, Y., Liang, Z., Chua, T.-S., Chua, Z.L., 2022. Shadewatcher: recommendation-guided cyber threat analysis using system audit records. In: *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 489–506.
- Zhu, T., Yu, J., Xiong, C., Cheng, W., Yuan, Q., Ying, J., Chen, T., Zhang, J., Lv, M., Chen, Y., et al., 2023. Aptshield: a stable, efficient and real-time apt detection system for linux hosts. *IEEE Trans. Dependable Secure Comput.*
- Zimba, A., Chen, H., Wang, Z., Chishimba, M., 2020. Modeling and detection of the multi-stages of advanced persistent threats attacks based on semi-supervised learning and complex networks characteristics. *Future Generat. Comput. Syst.* 106, 501–517.

Zhengqiu Weng received the Ph.D. degree in computer science and technology from the Zhejiang University of Technology, Zhejiang, China, in 2023, and the M.S. degree in software engineering from the Beijing Institute of Technology, Beijing, China, in 2005. She is also a professor with the Wenzhou University of Technology, Zhejiang, China. Her current research interests include networks security and big data technologies.

Weinuo Zhang is currently pursuing the bachelor degree in big data technologies from the Wenzhou University of Technology, Zhejiang, China. His research interests include system security, software security, and firmware analysis.

Tiantian Zhu received the PhD degree in computer science from Zhejiang University, Hangzhou, China, in 2019. He is currently an associate professor with the college of computer science and technology, Zhejiang University of Technology, China. His research interests include mobile security, system security and artificial intelligence.

Zhenhao Dou is currently pursuing the bachelor degree in big data technologies from the Wenzhou University of Technology, Zhejiang, China. His research interests include system security, software security, and firmware analysis.

Zhanxiang Ye received the M.S. degree from Wuhang University of in Wuhang, china, in 2006, He is currently a Associate professor in Wenzhou Polytechnic. His research interests include Network Technology and Information Security.

Haofei Sun is currently pursuing the M.S. degree in computer science and technology from the Zhejiang University of Technology, Zhejiang, China. His research interests include system security, software security, and firmware analysis.

Ye Tian is currently pursuing the M.S. degree in computer science and technology from the Zhejiang University of Technology, Zhejiang, China. His research interests include system security, software security, and firmware analysis.